Discussion 2 Linear vs Binary

Discussion Topic:

How do different search algorithms (linear vs. binary) impact user experience in modern technology applications, and when should developers choose one over the other? Use a specific example from a real application to illustrate your point.

My Post:

Hello Class,

Linear and binary searches have very different approaches to locating values within a data structure.

The linear search is a sequential searching algorithm that examines each element in a collection one by one, from start to finish, comparing it against the value being searched until this value is found or every element has been checked (TutorialsPoint, n.d.a). The data does not need to be sorted; in other words, the data collection does not need to be processed for the search to work as intended, making its implementation simple. However, the linear search algorithm has a time complexity of O(n). This implies that the time comparisons are directly proportional to the size of the dataset. Meaning, if the size of the data set doubles, the number of comparisons the algorithm must perform also doubles (CSU Global, n.d.). Its best-case scenario has an O(1) time complexity, that is, when the first element in the data set is the value searched.

On the other hand, the binary search is a divide-and-conquer search algorithm. The algorithm requires the data to be sorted beforehand, as it begins the search by comparing the searched value to the middle element within the data set, then if the values do not match, it discards an entire half of the collection based on a comparison that determines whether the target value is greater or less than the middle element, and repeats the process on the remaining half (TutorialsPoint, n.d.b). The binary search algorithm has a time complexity of O(log n). For example, searching a 15-element sorted array for the value 65 required only three comparisons with binary search, whereas linear search would have needed 10 (CSU Global, n.d.). This allows the search algorithm to be very efficient with exceptionally large datasets; for instance, for a dataset of one million elements, binary search needs roughly 20 comparisons, while linear search could require up to one million.

Real-world examples of these algorithms that can affect user experience can be found in social media applications. A social media platform can have millions of posts stored in a database. For instance, Instagram relies heavily on optimized search algorithms to deliver content instantly with minimal loading screens. The application uses PostgreSQL for its backend to manage data for over a billion users. Additionally, every time a user opens the app and loads their feed, the system needs to retrieve posts sorted by date, relevance, and connections from a set of tables containing billions of rows. This translates to the application needing to perform millions of concurrent queries in multiple databases (ByteByteGo, 2025).
To handle this colossal amount of data queries and, at the same time, maintain an enjoyable user experience, Instagram uses a B-Tree index, which is a generalization of binary search. PostgreSQL itself uses B-Tree indexes by default for nearly all lookups (PostgreSQL Documentation, n.d.). In

practice, PostgreSQL performs a binary search, using indexed columns, within each tree node, starting from the root to the leaf in logarithmic time (date) to locate the exact data needed (Huda, 2025). It is important to note that without an index, the database must resort to a sequential scan, which is essentially a linear search through every row in the table.

When choosing between a linear and binary search, it is important to consider each search algorithm's advantages. For instance, binary search has an efficiency advantage; linear search, on the other hand, is an appropriate choice for several common situations. Linear search works well on unsorted data without needing any preprocessing; that is, it performs well on small datasets where the overhead of sorting would be an overreach, and it is the 'natural' approach for linked lists that do not support random access (Wikipedia, 2025). For example, scanning a short configuration file, validating form input using a small list of values, or searching through unsorted logs. Binary search, on the other hand, is very efficient in handling large and sorted data sets, where data retrieval is frequently needed.
Ultimately, the choice between linear and binary search is a trade-off determined by the size of the dataset, the frequency of queries, and whether the data requires constant updating and scaling.

-Alex

References:

ByteByteGo. (2025, February 18). How Instagram scaled its infrastructure to support a billion users. *https://blog.bytebytego.com/p/how-instagram-scaled-its-infrastructure*

CSU Global. (n.d.). *Module 2: Searching and Algorithm Analysis* [Interactive lecture]. CSC506 - Design and Analysis of Algorithms. Canvas.

Huda, M. (2025, February 26). B-Tree implementation in PostgreSQL: Deep dive into database indexing. Medium. *https://iniakunhuda.medium.com/b-tree-implementation-in-postgresql-deep-dive-into-database-indexing-b1a34032637d*

PostgreSQL Documentation. (n.d.). Index types. *https://www.postgresql.org/docs/current/indexes-types.html*

TutorialsPoint. (n.d.a). Linear search algorithm. *https://www.tutorialspoint.com/data_structures_algorithms/linear_search_algorithm.htm*

TutorialsPoint. (n.d.b). Binary search algorithm. *https://www.tutorialspoint.com/data_structures_algorithms/binary_search_algorithm.htm*

Wikipedia. (2025). Binary search. *https://en.wikipedia.org/wiki/Binary_search*